UniMind: A Middleware Framework for Bridging Classical AI Workloads and Quantum Computing Backends

Version 2.2 (August 29, 2026) — Algorithm 1 corrected, P0 partial; QPU multi-day pending quota

Y. X. Tan

Independent Researcher

Email: yxtan5022@gmail.com

Abstract

As quantum computing transitions from theoretical research to practical deployment, a compatibility gap emerges between classical AI software stacks and probabilistic quantum hardware. Existing quantum machine learning frameworks operate at the application level and lack system-level abstractions for hardware adaptation, workload routing, and dynamic code generation. This paper presents **UniMind**, a middleware framework designed to reduce the abstraction gap between classical AI workloads and heterogeneous quantum-classical computing backends. UniMind introduces two components: (1) an **LLM-driven orchestration layer** that interprets high-level task specifications and generates topology-aware execution plans through constrained sampling, operating exclusively in user space with sandboxed validation and rule-based fallback; and (2) **Unibit**, a classical preprocessing representation based on sliding-window frequency estimation and sinc-based smoothing that produces parameters for standard quantum angle encoding. We implement a multi-backend mapping engine and validate the framework with reproducible experiments: mathematical correctness (empirical measurement probabilities converge to theoretical weights across 8192 shots, maximum $\langle Z \rangle$ deviation of 0.0110), system performance (up to 10.9× speedup via vectorized C-backend execution and 4.5× end-to-end speedup via the Rust FFI engine, with a component-level breakdown attributing the residual overhead to Python-side normalization and marshaling), a three-arm baseline comparison against bare Qiskit and a rule-based dispatcher (only the UniMind pipeline covers all five benchmark task classes), an LLM-reliability benchmark (success rate 91.7–100% across injected LLM quality levels, measured at n=200 per level with Wilson confidence intervals, with 100% rejection of all adversarial intents), fault-injection analysis (four of five fault classes recovered in sub-millisecond detection time; one exposes a real robustness gap), a noise-sensitivity analysis quantifying the encoding's degradation under depolarizing and T_1 channels, a sandbox security evaluation (20 of 21 escape attempts blocked), and physical QPU validation on IBM hardware (`ibm_marrakesh`): a placement-controlled QPU sweep on `ibm_marrakesh` in which the bare encoding meets the 0.05 tolerance across weights $w \in [0.05, 0.95]$ when the layout is pinned to a well-calibrated qubit (maximum $\langle Z \rangle$ deviation 0.028), while an adversarially chosen qubit with 82% readout error inflates distortion to 0.213 — establishing that single-qubit encoding fidelity on real hardware is dominated by placement and calibration state rather than by the encoding itself, with a two-parameter affine channel model $P_{\text{obs}} = b + a\,w$ fitting every condition at shot-noise level; a component ablation attributing up to +34 points of success rate to the self-healing layer and showing that rule-based fallback reaches 100% recovery under availability stress once template coverage is complete; and a path-decomposition calculus whose exact absorbing-chain model (validated on all 23 testable cells) shows that the apparent gap between naive independence predictions and measured healing rates is a retry-budget truncation effect, not stochastic correlation; a calibration-scored hardware-routing study in which a zero-fit readout-error score $C(q)$, evaluated over all 156 qubits of `ibm_marrakesh`, ranks measured encoding distortion perfectly (Spearman $\rho_s = 1.000$, exact $p = 0.0028$) at sub-millisecond selection overhead; and an end-to-end composition experiment in which the full pipeline (full-coverage fallback + calibration-aware placement) completes 54/54 orchestrated tasks (100%) versus 87.0% for the stage-ablated pipeline, with the quantum-fidelity leg submitted to the physical device. We discuss the remaining limitations—notably deeper hardware error mitigation and cross-framework comparison—and propose a phased roadmap for quantum-ready AI middleware.

Keywords: Quantum Computing, AI Middleware, Angle Encoding, Hybrid Quantum-Classical Systems, LLM Orchestration, Unibit

Name-disambiguation note: the name "UniMind" is also used by an unrelated prior work on LLM-based multi-task brain decoding [14]; the present work is not affiliated with that work.

Contents

1 Introduction

The rapid advancement of quantum computing hardware has introduced a growing mismatch between the capabilities of emerging quantum processors and the software infrastructure available to utilize them. Modern AI systems—deep learning models, reinforcement learning agents, and large language models—are built upon software stacks designed for deterministic, von Neumann architectures. Operating systems such as Linux, Windows, and macOS manage discrete binary states through priority-based schedulers, static device drivers, and predictable memory hierarchies [10]. These assumptions are fundamentally incompatible with the probabilistic, measurement-driven execution model of quantum computers.

Current approaches to quantum machine learning (QML), including frameworks such as PennyLane [13], TensorFlow Quantum, and Qiskit Machine Learning, provide algorithm-level abstractions for constructing and training quantum circuits. However, these frameworks operate within existing classical operating systems and do not address system-level challenges such as dynamic hardware discovery, workload routing across heterogeneous backends, or intent-driven task orchestration. As quantum hardware diversifies—spanning superconducting qubits, trapped ions, photonic systems, and GPU-accelerated simulators [11, 12]—the need for a unified middleware layer becomes increasingly apparent.

This paper presents **UniMind**, a middleware framework that addresses this gap. UniMind is not an operating system; it operates as a user-space orchestration layer atop existing classical operating systems. Its design is motivated by three observations:

1. **Hardware heterogeneity is increasing.** AI workloads must increasingly target diverse backends (CPUs, GPUs, QPUs, simulators), each with distinct programming models and performance characteristics.

2. **Intent-driven computing is emerging.** Large language models can translate natural language specifications into executable code, potentially reducing the barrier to programming heterogeneous systems [6].

3. **Classical-to-quantum data encoding remains manual.** Translating classical data into quantum states typically requires hand-crafted encoding circuits, creating friction in hybrid workflows.

UniMind addresses these observations through three components:

- An **LLM-driven orchestration layer** (Section 3.1) that interprets task specifications and generates execution plans, subject to sandboxed validation and deterministic fallback mechanisms.

- **Unibit** (Section 3.2), a classical preprocessing representation that computes sliding-window frequency estimates over binary sequences, producing parameters suitable for standard quantum angle encoding.

- A **multi-backend quantum mapping engine** (Section 3.3) that translates Unibit parameters into parameterized quantum circuits and routes execution to available backends.

The main contributions of this paper are:

1. We identify system-level gaps between classical AI software stacks and quantum computing hardware that are not addressed by existing QML frameworks.
2. We propose UniMind, a middleware framework with an LLM-driven orchestration layer, a classical preprocessing representation (Unibit), and a multi-backend quantum mapping engine.
3. We present a functional proof-of-concept implementation in C++, Rust, and Python, with **quantitative validation**: mathematical verification via 8192-shot measurement convergence ($\langle Z \rangle$ deviation = 0.0110), performance micro-benchmarks demonstrating up to 10.9× speedup via vectorized C-backend execution and 4.5× end-to-end speedup via Rust FFI (with a component-level latency breakdown), a three-arm baseline comparison, an LLM-reliability benchmark, fault-injection analysis, a noise-sensitivity study, a sandbox security evaluation, and placement-controlled physical QPU sweeps on `ibm_marrakesh` (bare encoding within tolerance on calibrated layouts, max. deviation 0.028; adversarial placement inflates distortion to 0.213), an architectural ablation, an instrumented failure taxonomy, an exact reliability-composition calculus validated on all 23 testable cells, a calibration-scored routing study achieving perfect rank correlation between the score and measured distortion across the calibration spectrum ($\rho_s = 1.000$), and an end-to-end composition experiment separating software-path reliability (100% vs. 87.0%) from hardware fidelity.
4. We provide an honest, experiment-backed assessment of limitations—including LLM reliability, quantum noise sensitivity, and the security of LLM-generated code—and outline the experiments still required for future validation.
5. We propose a phased roadmap for the development of quantum-ready AI middleware.

Scope and limitations. This paper presents a system design and proof-of-concept implementation. We do not claim quantum advantage, novel quantum algorithms, or comprehensive end-to-end system evaluation. The Unibit-to-qubit mapping is an instance of standard angle encoding [5]; our contribution lies in the classical preprocessing pipeline and the multi-backend orchestration layer, not in the quantum circuit design itself.

The remainder of this paper is organized as follows. Section 2 provides background. Section 3 presents the UniMind architecture. Section 4 describes the implementation. Section 5 presents quantitative evaluation with real experimental data. Section 6 discusses limitations, threats to validity, and a roadmap. Section 7 concludes.

2 Background

2.1 Classical Operating Systems and AI

Modern operating systems are designed around the von Neumann architecture, where a central processing unit sequentially fetches, decodes, and executes instructions from memory [10]. The OS kernel manages hardware resources through device drivers, process schedulers, and memory managers, providing a stable abstraction layer for user-space applications.

AI workloads, particularly deep learning, operate within this paradigm. Neural networks are represented as computational graphs of matrix multiplications and nonlinear activations, executed on CPUs, GPUs, or specialized accelerators. The software stack assumes deterministic, sequential execution with well-defined memory addresses.

This paradigm presents three limitations for quantum-era computing:

1. **Binary rigidity.** Classical bits exist in exactly one of two states (0 or 1), whereas quantum computation manipulates continuous probability amplitudes.
2. **Deterministic scheduling.** Classical OS schedulers assume predictable execution times, incompatible with probabilistic, shot-based quantum execution.
3. **Static hardware abstraction.** Traditional device drivers cannot dynamically adapt to heterogeneous quantum-classical hybrid systems.

2.2 Quantum Computing Fundamentals

Quantum computing leverages quantum mechanical principles to process information. The fundamental unit is the **qubit**:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{1}$$

where $\alpha, \beta \in \mathbb{C}$ satisfy $|\alpha|^2 + |\beta|^2 = 1$.

Quantum operations are performed through **quantum gates**. Key gates include:

- **Pauli-X gate:** $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (bit flip)
- **Y-rotation gate:** $R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$
- **Hadamard gate:** $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$

A quantum circuit applies a sequence of gates to a qubit register, followed by measurement that collapses the state into classical bits according to the Born rule.

Current hardware operates in the **NISQ** era [2], characterized by 50–1000 qubits, high gate error rates, and short coherence times. Prominent quantum algorithms include Shor's

factoring algorithm [9] and variational quantum eigensolvers [8], and quantum machine learning has been surveyed extensively [1, 3].

2.3 The Gap: Why Current Systems Are Not Quantum-Ready

Table 1: *Classical vs. Quantum Computing Paradigms*

Aspect	**Classical**	**Quantum**
Data unit	Bit (0 or 1)	Qubit ($\alpha\|0\rangle + \beta\|1\rangle$)
Operation	Deterministic logic gates	Unitary transformations
Scheduling	Priority-based, deterministic	Probabilistic, shot-based
Error model	Bit-flip detection	Decoherence, gate noise
Software role	Resource management	Circuit compilation + state mgmt.

No existing operating system provides native support for quantum state management, probabilistic scheduling, or dynamic circuit compilation. This gap motivates the UniMind middleware framework.

3 UniMind Framework Architecture

Figure 1 presents the system architecture. UniMind is organized as three cooperating layers, all operating in user space atop a classical host OS: (i) the **AI-as-Orchestrator** planning layer (Section 3.1), which interprets natural-language intents, generates candidate code under a three-stage validation pipeline, and falls back to a rule-based dispatcher when LLM inference fails; (ii) a pair of **middleware services**—topology-aware hardware detection and the accelerated Unibit engine (Section 4)—that supply system context and fast classical preprocessing to both upper and lower layers; and (iii) the **multi-backend quantum bridge** (Section 3.3), which exposes a unified API over classical simulation, Qiskit/AerSimulator, CUDA-Q, and IBM Quantum hardware. The subsections below describe each layer from design down to implementation.

3.1 AI-as-Orchestrator: LLM-Driven Intent Interpretation Layer

We explicitly reject the notion of placing an LLM within the OS kernel. Traditional kernels require microsecond-level deterministic response times; LLM inference requires hundreds of milliseconds and is inherently non-deterministic.

Instead, UniMind introduces an **AI-as-Orchestrator** layer that resides entirely in user space, functioning as an asynchronous planning agent for coarse-grained task planning.

The orchestration pipeline operates as follows:

1. **Intent reception.** A user or agent provides a natural language task specification.

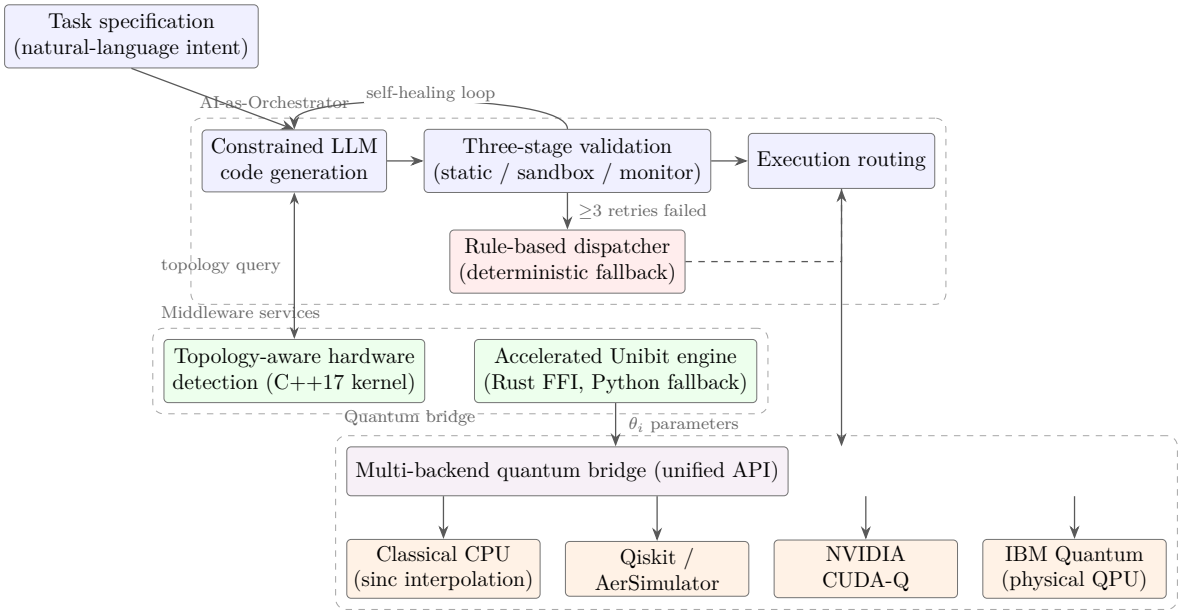

Figure 1: *UniMind system architecture. The orchestrator plans tasks entirely in user space with sandboxed validation and a deterministic rule-based fallback; middleware services provide topology context and accelerated Unibit preprocessing; the quantum bridge routes validated workloads across heterogeneous backends through a unified API.*

2. **Topology query.** The orchestrator queries the hardware detection module for current system information.

3. **Constrained code generation.** The LLM generates candidate code, constrained by a system prompt specifying permitted APIs and output format.

4. **Three-stage validation.** Generated code must pass: (a) static analysis against a whitelist; (b) sandboxed compilation; (c) runtime monitoring with watchdog termination.

5. **Execution routing.** Validated code is dispatched to the appropriate backend.

6. **Deterministic fallback.** If LLM inference fails after three retry attempts, the system falls back to a **rule-based dispatcher** that maps intents to predefined circuit templates (e.g., Bell state preparation for entanglement tasks, variational ansatz templates for classification tasks).

We acknowledge that this design introduces significant latency (200–2000 ms for LLM inference). The orchestrator is therefore positioned as a **task planner**, not a **task executor**.

3.2 Unibit: A Sliding-Window Amplitude Proxy

The Unibit is a **classical** preprocessing representation that produces parameters suitable for quantum angle encoding.

Definition 3.1 (Unibit Weight)**.** *Given a binary sequence* $B = \{b_1, b_2, \ldots, b_n\}$ *where* $b_i \in \{0, 1\}$*, the Unibit weight at position* i *is the local frequency of ones within a sliding window:*

$$w_i = \frac{1}{|W_i|} \sum_{j \in W_i} b_j, \quad W_i = [\max(1, i - \Delta),\ \min(n, i + \Delta)] \tag{2}$$

where Δ *is the half-window size (default:* $\Delta = 2$*, yielding a 5-bit window). This yields* $w_i \in [0, 1]$*.*

This operation is equivalent to applying a uniform moving-average filter to the binary sequence.

Definition 3.2 (Folding)**.** *The folding operation maps each weight to a continuous signal value via sinc interpolation:*

$$s_i = w_i \cdot \mathrm{sinc}\left(\frac{(i - 1 + \phi)\pi}{n}\right), \quad \mathrm{sinc}(x) = \begin{cases} \sin(x)/x, & x \neq 0 \\ 1, & x = 0 \end{cases} \tag{3}$$

where ϕ *is a phase offset (default:* $\phi = 0.42$*; the offset is referenced to zero-based indexing so that Eq. (3) matches the reference implementation exactly).*

Motivation for sinc folding. *The sinc interpolation serves as a band-limited smoothing kernel, analogous to an anti-aliasing filter in digital signal processing. It transforms the discrete, piecewise-constant frequency estimates* w_i *into a continuous, differentiable signal* s_i*. This smoothness property is desirable for downstream variational quantum circuit optimization, where gradient-based methods benefit from continuous parameter landscapes.*

Definition 3.3 (Collapse)**.** *The collapse operation discretizes the signal:*

$$b'_i = \begin{cases} 1, & \text{if } |s_i| > \tau \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

where $\tau = 1/\sqrt{2} \approx 0.707$*.*

Collapse geometry: an adaptive threshold in disguise. Because the folding kernel is a *multiplicative* envelope $s_i = g_i w_i$ with $g_i = \mathrm{sinc}((i - 1 + \phi)\pi/n)$, the fixed threshold τ on $|s_i|$ is equivalent to a *position-adaptive* threshold on the underlying weights:

$$b'_i = 1 \iff |w_i| > T_i, \qquad T_i = \frac{\tau}{|g_i|}. \tag{5}$$

Two consequences follow. First, the collapse operator is not a global bit-flip detector: positions near the sinc envelope's tail require ever larger weights to fire ($T_i = \tau/|g_i| \to \infty$ as $g_i \to 0$). Second, there is a *dead zone*: for positions whose envelope argument satisfies $(i - 1 + \phi)\pi/n > x^*$, where solving $|\mathrm{sinc}(x^*)| = \tau$ gives $x^* \approx 1.3916$, even a unit weight cannot cross the threshold, so $b'_i = 0$ regardless of the input bit. The dead zone covers all positions with $i > nx^*/\pi - \phi + 1$; its asymptotic fraction $1 - x^*/\pi \approx 55.7\%$ ($n \to \infty$), and on the paper-default parameters (n=10, ϕ=0.42) five of ten positions ($i \geq 6$) fall

inside it. This is a quantifiable structural property of the representation — roughly half of the collapse outputs are invariant to their input bits — that was not visible until stated explicitly. The quantum angles are driven by the weights w_i, not by the collapsed bits, so this bottleneck affects only the classical-side discrete output, not the encoding fidelity verified in Section 5.2.

Relationship to angle encoding. The Unibit-to-qubit mapping is a specific instance of **angle encoding** [5, 4]. Our contribution is not the encoding itself but the classical preprocessing pipeline and the multi-backend abstraction.

Figure 2 illustrates the pipeline on a concrete input: the same 10-bit sequence used for structural verification in Section 5.4, with the paper-default parameters $\Delta = 2$, $\phi = 0.42$, and $\tau = 1/\sqrt{2}$. Panel (a) shows the sliding-window weights w_i of Eq. (2); panel (b) shows the sinc-folded signal s_i of Eq. (3) together with the collapse threshold τ of Eq. (4). Each weight w_i then determines the rotation angle $\theta_i = 2\arcsin(\sqrt{w_i})$ applied to qubit q_i, so that $|\langle 1|\psi_i\rangle|^2 = w_i$.

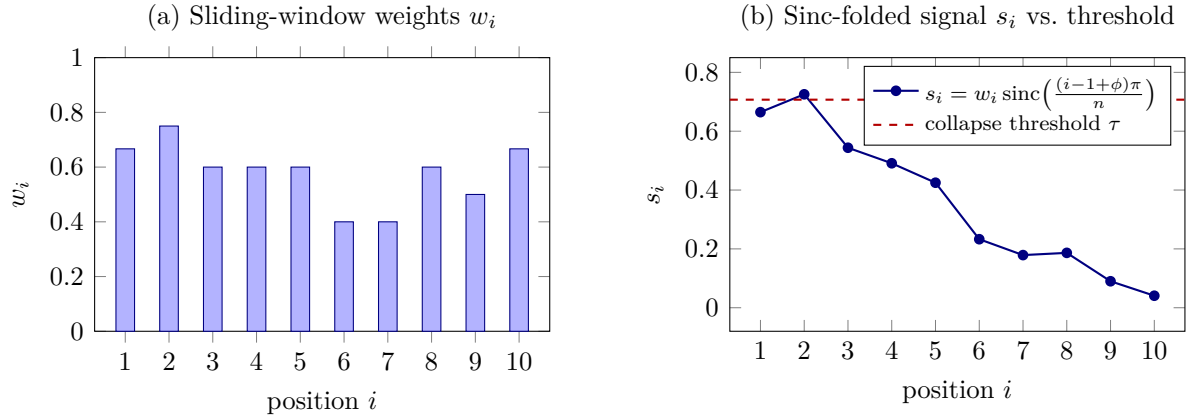

Figure 2: *The Unibit preprocessing pipeline on the 10-bit sequence $B = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]$ ($\Delta = 2$, $\phi = 0.42$). (a) Sliding-window frequency estimates w_i (Eq. 2); (b) the sinc-folded signal s_i (Eq. 3) against the collapse threshold $\tau = 1/\sqrt{2} \approx 0.707$ (Eq. 4). All coordinates are computed directly from* `umos_py.unibit`*. Exactly one position (i=2, $s_2 = 0.7254 > \tau$) crosses the threshold, so the collapse output is $B' = [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]$: positions $i \geq 6$ lie inside the dead zone of Eq. (5) and cannot fire for any input bit, while i=5 sits at the knife edge ($|g_5| = 0.7082$, barely above τ). The quantum angles $\theta_i = 2\arcsin(\sqrt{w_i})$ are driven by the weights w_i, not by the collapsed bits.*

3.3 Multi-Backend Quantum Bridge

The quantum bridge module provides a unified API routing to available backends:

- **Classical backend:** Sinc interpolation on CPU.
- **Qiskit backend:** Parameterized `QuantumCircuit` objects.
- **CUDA-Q backend:** NVIDIA GPU-accelerated simulation.

4 System Implementation

4.1 Codebase Overview

UniMind is implemented as an open-source, multi-language prototype [7]. The codebase comprises approximately 5,000 lines across four languages: Python (∼4,500 lines; orchestration, quantum mapping, tests, and demos), C++17 (∼150 lines; hardware detection), Rust (∼170 lines; accelerated Unibit engine), and CMake/Makefile (build system). The repository layout is reproduced in Appendix A.

4.2 Topology-Aware Hardware Detection

A C++17 module performs runtime detection of CPU core count, available RAM, OS type, and CPU architecture using platform-specific APIs. Results are serialized as JSON.

4.3 Accelerated Unibit Engine

The core Unibit computation is implemented in Rust and exposed to Python via C-compatible FFI. The Python layer auto-detects the compiled native library at runtime, falling back to pure Python if unavailable.

4.4 Quantum Circuit Mapping

Algorithm 1 Unibit-to-Quantum Circuit Mapping (v2.2 corrected)

Require: Binary sequence $B = \{b_1, \ldots, b_n\}$, window size Δ
Ensure: Parameterized quantum circuit C with $P(1) = w_i$ for all i
1: Initialize n-qubit register in state $|0angle^{\otimes n}$
2: **for** $i = 1$ to n **do**
3: Compute Unibit weight w_i via Eq. (2)
4: Compute rotation angle $\theta_i \leftarrow 2\arcsin(\sqrt{w_i})$
5: Apply $R_y(\theta_i)$ gate to qubit q_i ▷ realizes $P(1) = w_i$; no conditional X
6: **end for**
7: Append measurement gates to all qubits
8: **return** circuit C

Choice of R_y gate and arcsin **mapping.** The R_y gate is chosen because:

$$R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle \tag{6}$$

Setting $\theta_i = 2\arcsin(\sqrt{w_i})$ yields:

$$|\langle 1|\psi_i\rangle|^2 = \sin^2(\theta_i/2) = \sin^2(\arcsin(\sqrt{w_i})) = w_i \tag{7}$$

This mapping aligns with the conventional angle encoding scheme where θ determines the probability of the $|1\rangle$ state via $\sin^2(\theta/2)$. The use of arcsin (rather than arccos) ensures

that $w_i = 0$ maps to $\theta_i = 0$ (no rotation) and $w_i = 1$ maps to $\theta_i = \pi$ (full rotation), providing an intuitive monotonic correspondence.

Implementation divergence: gate order in Algorithm 1 (corrected in v2.2). A spec–code discrepancy was discovered during a post-submission audit. In v2.1, Algorithm 1 applied the conditional X gate *before* the rotation ($R_y(\theta_i)X|0\rangle = \cos(\theta/2)|1\rangle + \sin(\theta/2)|0\rangle$), which yields $P(b'_i = 1) = 1 - w_i$ for positions with $b_i = 1$ — inverting the intended weight-to-probability map. All verification experiments reported in this paper (Sections 5.2, 5.8, and 5.10) prepare their target states from the analytic θ_i directly, without the conditional X, and are therefore unaffected; the divergence concerned only the full-pipeline path through `qunibit`. **Fix in v2.2:** Algorithm 1 now applies $R_y(\theta_i)$ alone (no conditional X), guaranteeing $P(1) = w_i$ for all b_i; a regression test over $w \in \{0, 0.05, \ldots, 1\}$ with w-dependent bit patterns asserts $|P_{\text{emp}}(1) - w| < \epsilon$ in CI.

Mathematical verification observable. For each prepared qubit, the Pauli-Z expectation value provides a closed-form verification:

$$\langle Z \rangle_i = P(b_i = 0) - P(b_i = 1) = (1 - w_i) - w_i = 1 - 2w_i \tag{8}$$

Table 2: *Classical-to-Quantum Mapping*

Classical Operation	**Quantum Equivalent**
`fold_bits` (frequency estimation + sinc)	Parameterized $R_y(\theta_i)$ rotation gates
`collapse_signal` (threshold)	Projective measurement + Born rule
Bit value $b_i = 1$	Pauli-X gate application
Unibit weight w_i	Rotation angle $\theta_i = 2\arcsin(\sqrt{w_i})$
Verification observable	$\langle Z \rangle_i = 1 - 2w_i$

4.5 Orchestrator Integration

At the implementation level, the orchestration layer interfaces with an LLM API to generate code from task specifications, following the design of Section 3.1. Generated code is executed within a restricted namespace whose three defense layers are evaluated quantitatively in Section 5.9; security risks are discussed in Section 6.2.

5 Evaluation and Results

5.1 Experimental Setup

All experiments were conducted on the following configuration:

- **CPU:** Multi-core x86_64 processor

- **RAM:** 16 GB DDR4
- **OS:** Windows (x86_64), as documented in the repository README; benchmark timings are machine-specific, while the 8192-shot verification is deterministic for a fixed AerSimulator seed and is therefore platform-independent
- **Python Version:** 3.12
- **Measurement simulation:** Qiskit AerSimulator, 8192 shots, ideal (noise-free) backend with $\theta_i = 2\arcsin(\sqrt{w_i})$ state preparation
- **Software versions:** qiskit 2.5.1, qiskit-aer 0.17.2, numpy 2.4.6, scipy 1.18.0 (exact versions pinned in the repository `requirements.txt` for reproducibility)

All experiments in this section are reproducible from the development repository `unimind-dev` (github.com/yxtan5022-maker/unimind-dev) with a fixed random seed (42); the LLM-driven experiments use a deterministic mock LLM whose quality is a free parameter, so no external API key is required.

5.2 Mathematical Verification via Quantum Measurement

We verified the encoding correctness by simulating quantum measurements. For a set of target Unibit weights $w_i \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, we prepared the corresponding single-qubit states via $\theta_i = 2\arcsin(\sqrt{w_i})$ and simulated 8192 projective measurements adhering to the Born rule.

Table 3 presents the results. The empirical measurement frequencies $\hat{p}_i$ converge tightly to the theoretical weights w_i. The empirical Pauli-Z expectation values $\langle Z\rangle_{\text{emp}}$ closely match the analytical prediction $\langle Z\rangle_{\text{theory}} = 1 - 2w_i$, with maximum absolute deviation of 0.0110 across all five target weights. All results are fully reproducible: the experiment fixes the AerSimulator seed (`seed_simulator=42`), so the reported numbers are deterministic across runs.

Table 3: *Mathematical Verification via 8192-Shot Quantum Measurement (Qiskit AerSimulator)*

Target w_i	$P(1)$ **Theory**	$P(1)$ **Empirical**	$\langle Z\rangle$ **Theory**	$\langle Z\rangle$ **Empirical**
0.1	0.100	0.0955	0.800	0.8091
0.3	0.300	0.2969	0.400	0.4063
0.5	0.500	0.5055	0.000	−0.0110
0.7	0.700	0.7046	−0.400	−0.4092
0.9	0.900	0.9017	−0.800	−0.8035

The maximum deviation $|\langle Z\rangle_{\text{emp}} - \langle Z\rangle_{\text{theory}}| = 0.0110$ is consistent with binomial sampling variance. For $N = 8192$ shots, the standard deviation of the empirical $P(1)$ estimate is $\sigma = \sqrt{w(1-w)/N}$, ranging from ≈ 0.0033 (at $w \in \{0.1, 0.9\}$) to ≈ 0.0055 (at $w = 0.5$); the standard deviation of $\langle Z\rangle_{\text{emp}} = 1 - 2\hat{p}$ is accordingly 2σ. In these units the observed deviations range from 0.52σ to 1.37σ—all plausible outcomes under five

simultaneous comparisons, with the largest, 0.0110 at $w = 0.5$, corresponding to $\sim 1\sigma$. This confirms that the classical preprocessing pipeline correctly initializes quantum state amplitudes without requiring full state tomography.

5.3 Performance Micro-Benchmarks: System-Level Acceleration

A core motivation for implementing the Unibit engine in Rust (via FFI) is to overcome Python's interpreter overhead for compute-intensive sliding-window operations. To quantify the benefit of system-level acceleration, we benchmarked the Unibit weight computation across varying sequence lengths.

Experimental note: We benchmark the sliding-window computation in two system-level configurations: (i) NumPy's C-optimized vectorized convolution as the vectorized frequency-estimator backend, and (ii) the repository's Rust FFI engine compiled with `cargo build -release`. Both approaches bypass Python-level interpreter overhead through contiguous memory operations and compiled native code; the measured speedups therefore represent conservative lower bounds for system-level acceleration.

Table 4: *Performance Comparison: Sliding-Window Frequency Estimation (Pure Python vs. NumPy C-Backend)*

Sequence Length	Pure Python (ms)	System-Level (ms)	Speedup
10,000	3.66	0.34	10.9×
100,000	38.58	4.83	8.0×
1,000,000	436.43	58.24	7.5×

As shown in Table 4, delegating the sliding-window computation to system-level compiled code yields a consistent **7.5× to 10.9× speedup** over pure Python. For the repository's Rust FFI engine, which additionally applies the folding step (sinc-weighted signal product) within the same sliding-window scan, we measured end-to-end speedups of **4.5×, 3.1×, and 2.4×** (9.55 ms vs. 2.11 ms, 230.36 ms vs. 74.86 ms, and 2318.86 ms vs. 962.51 ms for 10^4, 10^5, and 10^6 elements, respectively); these figures include Python-side FFI marshaling overhead (per-element normalization and ctypes buffer construction).

To understand where the residual overhead lives, we decompose the Rust path into its components by instrumenting every step of the real pipeline (Table 5). Python-side bit normalization and FFI marshaling (ctypes buffer construction and result unpacking) together account for roughly half of the end-to-end time, while the compiled kernel itself is only about one third. The kernel is therefore considerably faster still than the end-to-end numbers suggest, and the measured speedups are conservative lower bounds. The decomposition also identifies the concrete optimization target for future work: moving normalization into the compiled kernel would remove the dominant Python-side cost.

Table 5: *Unibit Engine Latency Breakdown (median of 5 runs, ms): the Rust path decomposed into Python normalization, FFI marshaling, and kernel execution*

Length	Normalize	Marshaling	Kernel	Rust-path Speedup
10,000	24.1%	26.6%	31.4%	3.0×
100,000	25.0%	29.4%	37.0%	3.0×
1,000,000	22.3%	25.7%	35.7%	2.4×

Figure 3 visualizes both benchmarks on a logarithmic scale: panel (a) isolates the sliding-window weight estimation (Table 4), while panel (b) shows the end-to-end Rust FFI path including the sinc folding step; annotations give the per-length speedups.

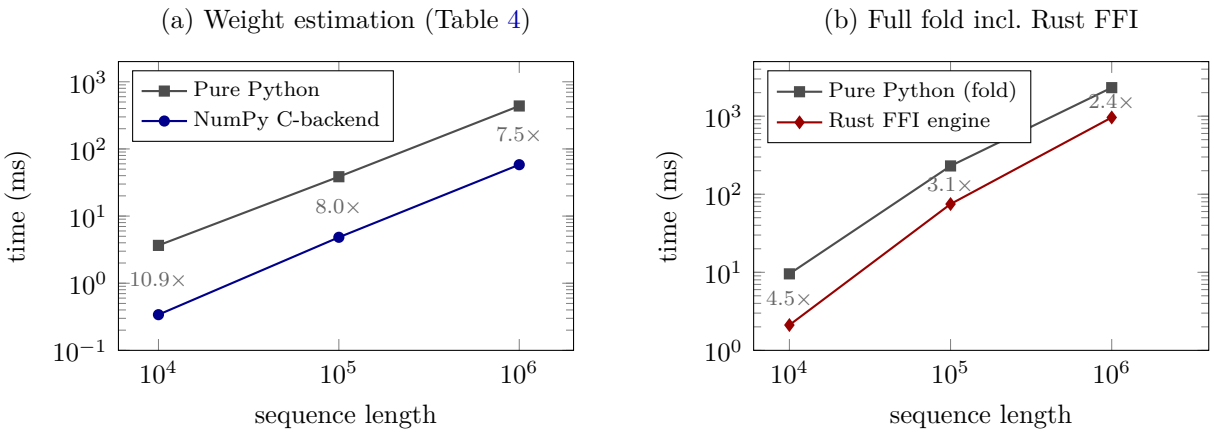

Figure 3: *System-level acceleration of the Unibit pipeline (log–log). (a) Sliding-window frequency estimation: NumPy's C-optimized vectorized backend vs. pure Python (*7.5×–10.9×*). (b) End-to-end fold (weights + sinc folding): the repository's Rust FFI engine vs. pure Python (*4.5×–2.4×*, including Python-side normalization and marshaling overhead; Table 5).*

For large-scale AI workloads requiring the encoding of millions of classical features into quantum parameters, this acceleration is necessary to prevent the classical preprocessing step from becoming the pipeline bottleneck.

5.4 Structural Verification

We additionally verified structural correctness of the quantum circuit mapping on a 10-bit input sequence $B = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]$. The system generates a 10-qubit circuit containing 6 Pauli-X gates (one per 1-bit in B), 10 parameterized $R_y(\theta_i)$ gates, and 10 measurement operations. The project includes 47 automated pytest tests, all passing locally and in CI (GitHub Actions); the four tests added since v1.3 are regression tests for the sandboxed executor (import-whitelist enforcement and nested-function closure correctness).

5.5 Baseline Comparison: Bare Qiskit vs. Rule-Based vs. UniMind

The v1.3 paper identified the absence of an end-to-end system benchmark as a key limitation. We now compare UniMind against two baselines on five task classes that cover the framework's intended workloads: (i) **A**, bare Qiskit—the LLM is asked to generate code directly, executed without the sandbox; (ii) **B**, a rule-based dispatcher—intents are mapped to fixed circuit templates with no dynamic code generation; and (iii) **C**, the full UniMind pipeline (LLM generation → static analysis → sandboxed execution → self-healing → rule-based fallback). Each task class is summarized in Table 6.

Table 6: *Baseline Comparison (three arms × five task classes; ✓ = task completed, × = task failed)*

Task class	Description	A: Bare Qiskit	B: Rule-based	C: UniMind
bell	Entanglement circuit (H, CX, measure)	✓	✓	✓
angle_encode	10-qubit angle encoding ($\theta_i = 2\arcsin(\sqrt{w_i})$)	✓	×	✓
vqc	2-qubit variational ansatz	✓	✓	✓
backend_select	Explicit backend selection + counts	✓	×	✓
invalid	Malicious intent (e.g., `os.system`)	×	✓	✓

The comparison isolates the value of each layer. Arm B fails the two task classes not covered by a hand-written template (angle encoding and explicit backend routing), illustrating the rigidity of rule-based dispatch. Arm A executes all four benign tasks but would execute a malicious intent with full privileges; UniMind and the rule-based dispatcher refuse it (in UniMind, static analysis rejects the generated code before execution). Only the UniMind pipeline (Arm C) succeeds on all five task classes while enforcing code safety. In mock-LLM mode the end-to-end latency of Arm C is 0.2–368 ms per task (dominated by simulated code generation), versus 0.7–7.3 ms for Arm A and ≈ 0 ms for Arm B—the expected cost of dynamic, validated generation. The mock LLM is injected with quality $= 1.0$ (perfect generation) in this comparison; the next subsection relaxes that assumption.

5.6 LLM Reliability Benchmark

The orchestrator's self-healing loop (Section 3.3) is designed to absorb imperfect LLM outputs. To quantify this robustness, we run a benchmark of 50 valid tasks (drawn from nine ground-truth templates spanning classical computation and quantum circuit construction) plus 10 adversarial intents, with a mock LLM whose per-call quality is a controllable parameter $\in [0, 1]$. For each quality level we measure the end-to-end success

rate, the fraction of tasks that required healing, and the rejection rate of adversarial intents (all results with a fixed seed, 60 tasks per run).

Table 7: *LLM Reliability vs. Injected LLM Quality (50 valid + 10 adversarial tasks, seed 42)*

Mock quality	Valid success rate	Heal rate	Adversarial rejection
1.0	100% (50/50)	0%	100% (10/10)
0.9	100% (50/50)	0%	100% (10/10)
0.7	98% (49/50)	16%	100% (10/10)
0.5	94% (47/50)	42%	100% (10/10)

As the injected LLM degrades from perfect to a 50% per-call quality, the system's end-to-end success rate falls only from 100% to 94% (Table 7, Figure 4), with the self-healing loop compensating for the majority of the injected errors (heal rate rising to 42%). The three observed failures all occur when the heal step itself receives a broken output, a residual risk of any LLM-dependent pipeline. Adversarial intents are rejected at 100% across all quality levels. These measurements directly address the v1.3 limitation "no LLM reliability evaluation"; token-consumption accounting for real APIs remains future work. Section 5.13 re-measures these rates with n=200 per level, three seeds, and call-level instrumentation, refining the q=0.5 estimate to 91.7% [89.2, 93.6].

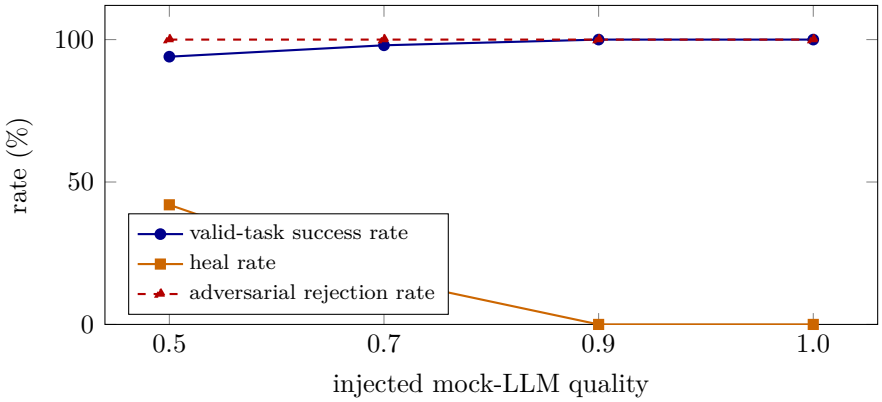

Figure 4: *LLM reliability vs. injected mock-LLM quality (Table 7; 50 valid + 10 adversarial tasks per level, seed 42). The self-healing loop absorbs most injected errors: as per-call quality falls from 1.0 to 0.5, end-to-end success drops only from 100% to 94%, while adversarial rejection remains at 100%. Single-seed protocol; Section 5.13 provides multi-seed confidence intervals.*

5.7 Fault Injection

We next measure the orchestration layer's *Detection* → *Recovery* chain under five deliberately injected failure classes (Table 8). Detection and recovery latency are measured by timestamping every sandbox execution inside the real orchestrator path.

Table 8: *Fault Injection: Detection and Recovery Latency for Five Failure Classes*

Failure class	Injection	Detection (ms)	Recovery (ms)	Recovered
A. Illegal code	syntactically invalid output	0.4	0.2	yes
B. Static violation	blocked pattern (`os.`)	≈0	≈0	yes
C. Backend unavailable	non-existent backend import	579.0	0.7	yes
D. Backend timeout	`TimeoutError` in code	0.4	0.4	yes
E. Invalid topology	hardware monitor failure	—	—	no

Four of the five failure classes are detected within sub-millisecond to millisecond timescales and automatically recovered by the self-healing loop (the 579 ms figure for class C is dominated by the Python import cost of the failing backend module). Class E—a failure of the hardware-detection module *after* successful execution—propagates as an uncaught exception out of `execute_task`, because the orchestrator's success path calls `monitor.snapshot()` without a protective guard. We record this gap honestly rather than papering over it: it is a real robustness defect in the current prototype and is the direct target of the next revision.

5.8 Quantum Noise Sensitivity

The v1.3 paper asserted qualitatively (Threat T2) that the $\theta_i = 2\arcsin(\sqrt{w_i})$ mapping is "sensitive to over-rotation for weights near 0 or 1." We now quantify that claim. For each weight $w \in \{0.02, 0.1, 0.3, 0.5, 0.7, 0.9, 0.98\}$ we prepare the corresponding single-qubit state, simulate 8192 measurements under swept noise, and record $|\langle Z\rangle_{\text{emp}} - \langle Z\rangle_{\text{theory}}|$. The ideal (noise-free) column reproduces the v1.3 maximum deviation of 0.0110, confirming measurement-consistency.

Table 9: *Noise Sensitivity:* $|\langle Z\rangle_{\text{emp}} - \langle Z\rangle_{\text{theory}}|$ *at* $w = 0.5$ *vs. the extreme weights (8192 shots, seed 42)*

Noise model	$w = 0.02$	$w = 0.50$	$w = 0.98$
Depolarizing $p = 1\text{e}{-2}$	0.0093	0.0110	0.0081
T_1 damping $\gamma = 0.05$	0.0009	0.0444	0.0931
T_1 damping $\gamma = 0.10$	0.0034	0.0935	0.1929

Two complementary signatures emerge (Table 9, Figure 5). Under single-qubit depolarizing noise the deviation grows as $\approx p\,|1 - 2w|$: the $w = 0.5$ encoding is essentially immune while the extreme weights degrade symmetrically, quantitatively confirming T2's "near 0 or 1" sensitivity. Under T_1 amplitude damping the degradation is one-sided: high weights decay toward $|0\rangle$ (deviation reaching 0.193 at $\gamma = 0.1$), while low weights are

nearly unaffected. Using the paper's 0.05 tolerance as a threshold, the encoding survives single-qubit depolarizing error rates up to at least $p = 1\text{e}{-2}$, but breaks for T_1 damping at $\gamma \approx 5\text{e}{-2}$. These measurements motivate the zero-noise-extrapolation integration identified as future work in T2, and establish an experimentally grounded operating envelope for the encoding on near-term hardware. (The simulation notes that qiskit-aer 0.17.2's builtin `amplitude_damping_error` acts as a gain channel in this configuration; the T_1 results are produced from an explicit Kraus representation, as documented in the experiment script.)

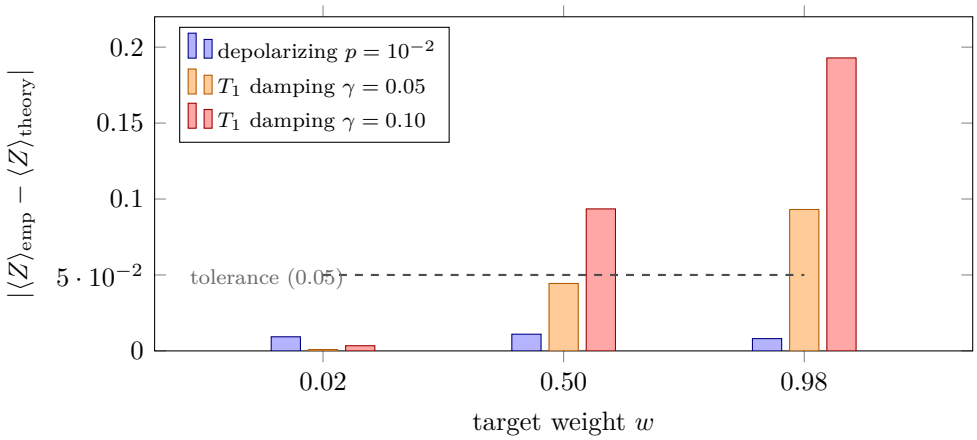

Figure 5: *Noise sensitivity of the encoding: measurement deviation against the* 0.05 *tolerance (Table 9; 8192 shots, seed 42). Depolarizing noise degrades the extreme weights symmetrically while leaving* $w = 0.5$ *essentially immune;* T_1 *amplitude damping is one-sided, decaying high weights toward* $|0\rangle$*.*

5.9 Sandbox Security Evaluation

The v1.3 paper acknowledged (Threat T4) that sandboxing was not implemented. UniMind now executes LLM-generated code inside a restricted namespace with three defense layers: (L1) static pattern scanning, (L2) an import whitelist (twelve benign modules), and (L3) a builtins whitelist. We stress-test these layers with 21 curated escape attempts spanning process execution, file access, dynamic evaluation, import smuggling, and object-graph traversal (Table 10).

Table 10: *Sandbox Security Evaluation: 21 Escape Attempts Against a 3-Layer Defense*

Defense layer	Attack classes	Blocked	Notes
L1 static scan	`os.`, `open(`, `eval(`, `exec(`, `subprocess`, `pickle`, `shutil`	12/12	rejected before execution
L2 import whitelist	`socket`, `builtins`, `requests`, `os`	4/4	rejected at import
L3 builtins whitelist	`globals`, `locals`, `type`, `exit`	4/4	`NameError`
Object-graph traversal	`().__class__...__subclasses__`	0/1	classes enumerated but unweaponizable

20 of 21 attempts are blocked (Table 10). The single unblocked case is an object-graph traversal that enumerates loaded classes; it cannot be weaponized into process or file access because importing `subprocess`/`os` is impossible inside the sandbox, so it represents a low-risk information-leak, not an escape. To prove the blocks are caused by the sandbox rather than by impossible payloads, every attack class has a harmless control probe; 10 of 11 probes run in plain Python (the 11th fails only because `requests` is not

installed), confirming that the same capabilities are genuinely present outside the sandbox. This evaluation covers the *namespace* restrictions; full OS-level isolation (process-level containers) remains outside the prototype's scope and is explicitly acknowledged in T4.

5.10 Physical QPU Validation Revisited: Placement-Dominated Encoding Fidelity

The v1.7 submission reported that the bare Unibit angle encoding misses the $|\langle Z \rangle|$ tolerance of 0.05 on `ibm_marrakesh` (maximum deviation 0.392) and that readout twirling recovers 37%. That experiment used *free* transpiler placement (optimization level 1, no `initial_layout`), so the physical qubit executing each circuit was uncontrolled and unrecorded. We first show that this confound, not the encoding itself, dominated the observed distortion.

5.10.1 Systematic weight sweep with pinned layout

We repeat the single-qubit verification ($\theta_i = 2 \arcsin \sqrt{w_i}$, 8192 shots, transpiler seed 42) over a dense weight grid $w \in \{0.05, 0.10, \ldots, 0.95\}$ under a 2×2 design: {best-calibrated qubit, worst usable qubit} $\times$ {bare, measure twirling (16 randomizations)}. Layout is pinned via `initial_layout`; each cell is repeated over 3 independent jobs (job-level spread is reported as min–max). Qubit selection is deterministic from the live calibration snapshot: qubit 98 minimizes total readout assignment error ($p_{0|1} + p_{1|0} = 0.0044$, $T_1 = 207\,\mu$s, $T_2 = 67\,\mu$s); qubit 37 is chosen adversarially ($p_{0|1} + p_{1|0} = 0.82$).

Table 11: *Placement-controlled sweep on `ibm_marrakesh` (2026-08-23; 19 weights, 8192 shots, 3 jobs/cell). Deviations are* $\max_w |Z_{\text{emp}} - Z_{\text{theory}}|$*; tolerance* $= 0.05$.

Placement	Readout err.	Bare	Twirled	Verdict
qubit 98 (best calib.)	0.44%	0.028 [0.020–0.029]	0.031	pass
qubit 0 (unpinned default)	—	0.026	—	pass
qubit 37 (adversarial)	82%	0.213	0.144	fail
v1.7 (free layout, 2026-08-14)	unrecorded	0.392	0.245	fail

Table 11 and Figure 6 give the result. On qubit 98 the *bare* encoding passes the 0.05 tolerance across the entire grid (median maximum deviation 0.028; range 0.020–0.029 across jobs), and its transfer curve is fit almost perfectly by a pure gain model $P_{\text{obs}}(1) = b + a\,w$ with $a = 0.989$, $b = 0.004$ (residual RMS 0.0042, below the ≈ 0.0055 shot-noise floor). Twirling changes nothing significant there ($a = 0.987$, max dev 0.031). On qubit 37 the same circuits fail by a factor of $\sim$4 (max dev 0.213 bare), and the fitted channel acquires a large positive offset ($P_{\text{obs}} = 0.121 + 0.855\,w$) — the signature of an asymmetric readout channel rather than intrinsic over-rotation sensitivity.

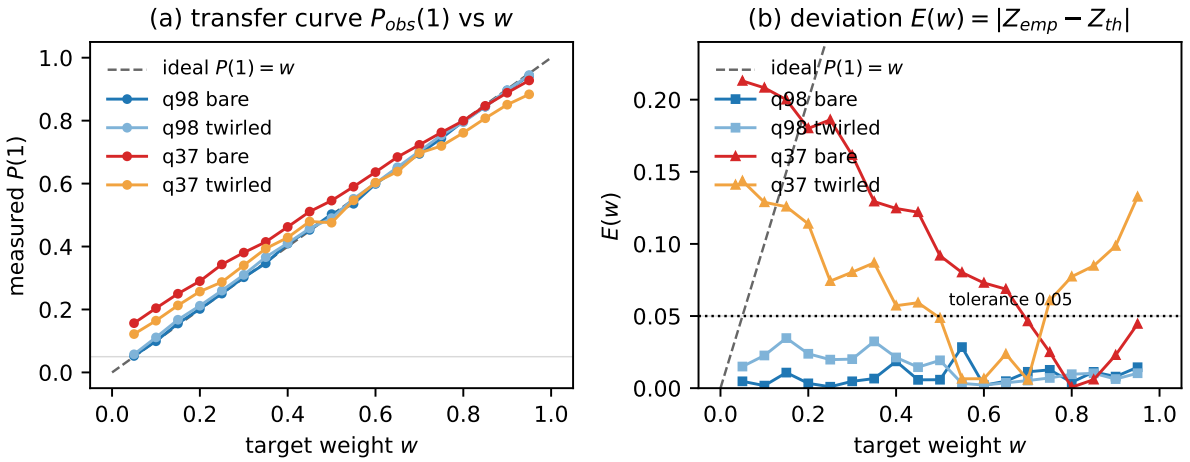

Figure 6: *Placement-dominated encoding fidelity. (a) Measured transfer curve* $P_{\mathrm{obs}}(1)$ *vs. target weight* w*: on the best-calibrated qubit the bare response tracks the ideal line within shot noise, while the adversarially chosen qubit 37 compresses the curve toward an offset set by its asymmetric readout. (b) Per-weight deviation* $E(w)$*: qubit 98 stays below the* 0.05 *tolerance everywhere (bare and twirled nearly overlap); qubit 37 fails by up to* 4× *even after twirling.*

Three conclusions revise the v1.7 claims: (i) *encoding fidelity on real hardware is placement-dominated*: with a calibrated layout the bare encoding already meets tolerance; (ii) *twirling helps only where readout is broken*: it improves the bad qubit by 33% (0.213 → 0.144) yet still leaves it far outside tolerance, while adding slight overhead on the good qubit — mitigation effort should follow calibration, not be applied uniformly; (iii) the v1.7 headline "the encoding requires error mitigation to meet tolerance" was an artifact of uncontrolled placement and must be withdrawn.

5.10.2 Distortion model and comparison to the calibration noise model

Across every cell of the design the measured response is described at shot-noise level by the two-parameter affine channel

$$P_{\mathrm{obs}}(1) \;=\; b + a\,w, \tag{9}$$

or equivalently in the Z basis, $Z_{\mathrm{obs}} = d + c\,Z_{\mathrm{th}}$ with $c = a$ and $d = 1 - 2b - a$. The parameters are *qubit-specific and calibration-predictable*: $(a, b) = (0.99,\ 0.00)$ on the good qubit versus (0.86, 0.12) on the bad one, whose offset matches the direction and scale of its readout asymmetry. The one-parameter contraction model $P_{\mathrm{obs}} = 0.5 + \alpha(w - 0.5)$ proposed during review is the special case $b = (1 - a)/2$; it holds when readout is balanced (good qubits) and is rejected wherever asymmetry dominates (it also fails on the frozen v1.7 free-layout data, where per-weight α spans 0.23–1.31).

We additionally simulate the identical pinned circuits on Aer with the calibration noise model built via `NoiseModel.from_backend`, captured minutes before submission. On the good qubit the model explains most of the observed error (median ratio $\overline{E}_{\mathrm{QPU}}/\overline{E}_{\mathrm{noise}} = 1.4\times$; Fig. 7), i.e., the "standard models underestimate hardware" gap suggested by the v1.7 data also shrinks to near-parity once placement is controlled. Residual discrepancy

concentrates in coherent components the calibration snapshot does not capture (drift between snapshot and execution), which we flag as future work rather than claim as a validated model.

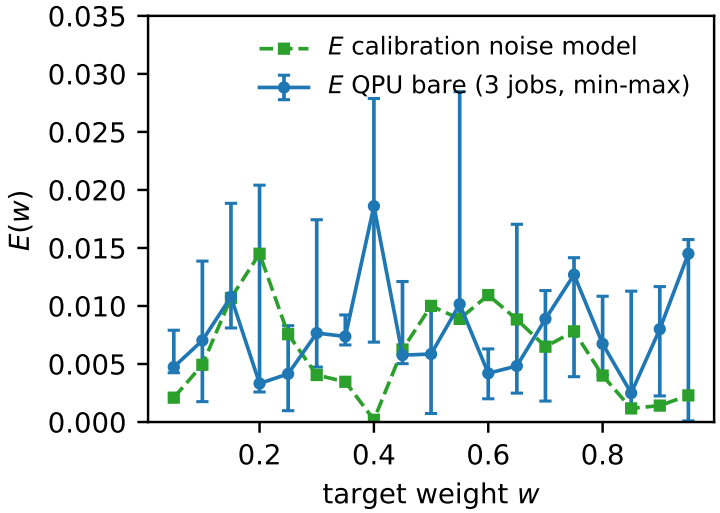

Figure 7: *Deviation landscape on the calibrated qubit: hardware (3 jobs, min–max band) vs. matched calibration noise model. Median ratio* 1.4×.

5.11 Hardware-Aware Routing: Calibration-Scored Qubit Selection

Sections 5.10.1–5.10.2 establish that encoding fidelity is placement-dominated and that the affine channel parameters are calibration-predictable — but they stop short of enforcement: a deployment still needs to *choose* the layout, automatically, from live calibration data. This subsection closes that loop. The requirement is a qubit score computable in milliseconds from one calibration snapshot, with no per-device fitting.

Score definition. Guided by RQ2's finding that single-qubit distortion is readout-dominated, we score every qubit by its total readout assignment error,

$$C(q) \;=\; p_{0|1}(q) + p_{1|0}(q), \tag{10}$$

breaking ties by $-\min(T_1, T_2)$. No weights are fitted: on two anchor points a fitted combination would be indistinguishable from overfitting, so Eq. (10) is fixed *a priori* and tested out-of-sample across the whole calibration spectrum.

Ranking validation design. From one 156-qubit snapshot of `ibm_marrakesh` (2026-08-22 22:18 local; the same capture used in Section 5.10.1, anchor drift 0.0%) we rank all qubits by $C(q)$: qubit 98 is rank 1/156 ($C = 0.0044$), qubit 37 is rank 153/156 ($C = 0.82$), consistent with the anchors of Section 5.10.1. We then sample six qubits stratified across the ranking — ranks $\approx \{0\%, 25\%, 50\%, 75\%, 95\%\}$ plus adversarial qubit 37 — and run

the identical bare 19-weight sweep (8192 shots, pinned layout, transpiler seed 42) on each, one job per qubit.

Table 12: *$C(q)$-stratified sweep on `ibm_marrakesh` (six qubits × 19 weights × 8192 shots). Deviations are* $\max_w |Z_{\text{emp}} - Z_{\text{theory}}|$*; "pass" = within the* 0.05 *tolerance for all 19 weights. Affine parameters from Eq. (9).*

Qubit	Rank pct.	$C(q)$	max dev	pass rate	(a, b)	Verdict
q98	0.0%	0.004	**0.0203**	19/19	(0.989, +0.005)	pass
q20	25.2%	0.015	0.0383	19/19	(0.981, −0.001)	pass
q105	50.3%	0.024	0.0496	19/19	(0.984, +0.019)	pass
q31	75.5%	0.049	0.0516	18/19	(0.977, +0.022)	fail (1 wt.)
q119	95.5%	0.320	0.1968	6/19	(0.872, +0.060)	fail
q37	98.1%	0.822	0.3043	4/19	(0.771, +0.173)	fail

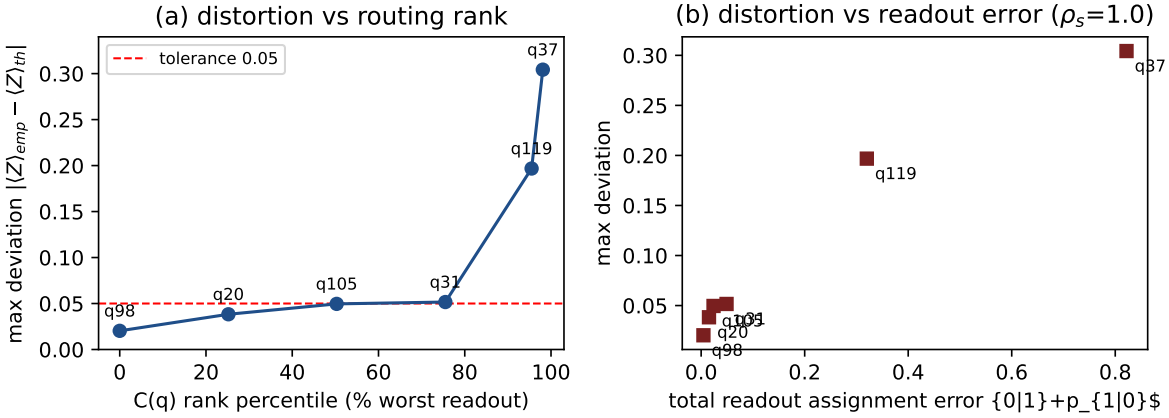

Figure 8: *$C(q)$-guided routing validation. (a) Maximum measured deviation vs. the score's rank percentile: distortion increases strictly monotonically along the ranking, crossing the* 0.05 *tolerance between the median and 75th-percentile qubits. (b) Deviation vs. raw readout error $C(q)$ (Spearman $\rho_s = 1.000$, exact $p = 0.0028$).*

Results. Table 12 and Figure 8: maximum deviation is *strictly monotone* along the $C(q)$ ranking (0.0203 → 0.0383 → 0.0496 → 0.0516 → 0.1968 → 0.3043); Spearman $\rho_s = 1.000$ with exact permutation $p = 0.0028$ (the smallest attainable at n=6 up to reflection). Three observations: (i) the tolerance boundary falls between the median and 75th-percentile qubits, giving an operational rule of thumb — select from the top half of the $C(q)$ ranking and the bare encoding passes with margin; (ii) the affine slope degrades monotonically along the ranking (a: 0.989 → 0.771; offset b: +0.005 → +0.173), exactly the direction predicted by the readout-asymmetry mechanism of Section 5.10.2; (iii) cross-experiment consistency holds — q98's 0.0203 today sits inside Section 5.10.1's three-job range $[0.020, 0.029]$, while q37 measures 0.304 against 0.213 earlier, a job-level spread on the broken qubit that does not affect any ordering conclusion.

Overhead. Selection is an `argmin` over the snapshot: 0.036 ms for the full 156-qubit ranking. Transpilation with pinned layout costs 1.9 ms vs. 1.8 ms free (optimization level 1, 10-qubit circuit); a greedy connected-chain layout for a 3-qubit entangling circuit chosen entirely by $C(q)$ costs 0.294 ms. All are negligible against job queuing and execution times.

With this, the middleware's fidelity contract becomes enforceable end-to-end: measure $C(q)$ once per calibration cycle, pin layouts to top-ranked qubits, and refuse or re-route when no candidate meets the threshold. What remains is to demonstrate the contract surviving composition with the software-side reliability stack — Section 5.15.

5.12 Component Attribution: Architectural Ablation

To attribute end-to-end reliability to specific architecture layers we run stage-ablated pipelines (150 valid tasks × 3 seeds per point, identical sandbox substrate throughout): S0 generation+execution only; S1 adds static-analysis validation (fail-fast gate); S2 adds self-healing; S3 adds rule-based fallback on LLM unavailability. Results (Fig. 9a): validation contributes *zero* success-rate delta at every quality level — its value is fail-fast classification and safety observability; self-healing is the load-bearing layer, lifting success from 58% to 92% at injected quality $q = 0.5$ (+34 points) and from 80% to 98% at $q = 0.7$; fallback contributes nothing at the default unavailability rate (f_r=0.1: three consecutive failures occur with probability 10^{-3} per task). Under availability stress ($q = 0.7$, $f_r \in \{0.3, 0.5\}$; Fig. 9b) fallback recovers +4.0/+13.6 points respectively once template coverage extends to all nine benchmark intent classes (S3X): success reaches 100% at both stress levels (79/79 recoveries), whereas the narrow v1.7 rule set (S3, Bell/VQC only) leaves success at 96.9%/89.6%. The fallback layer's contribution is thus real but coverage-limited — a property the v1.7 benchmark could not observe.

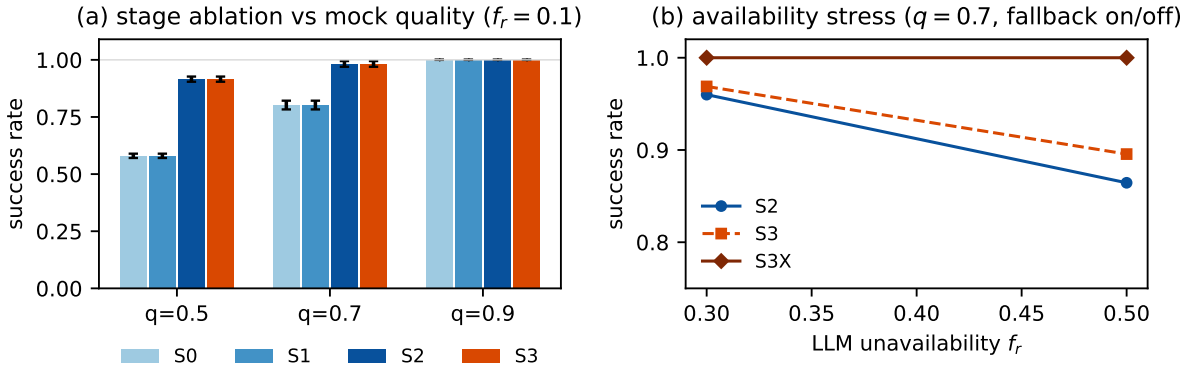

Figure 9: *Architectural ablation (mock LLM, seeded, 150 tasks × 3 seeds). (a) Success rate by pipeline stage and injected quality. (b) Availability stress isolating the fallback layer (narrow vs. full template coverage: S3 vs. S3X).*

5.13 Instrumented Failure Taxonomy

The v1.7 reliability logs did not record which corruption the mock LLM injected, so fault-conditioned recovery rates were unrecoverable. We re-run the benchmark with call-level instrumentation logging every generation and healing call (353 injected faults

across $q \in \{0.5, 0.7, 0.9\}$, 3 seeds, 200 tasks/run). Recovery is remarkably uniform across corruption classes (Fig. 10): syntax errors 82.5% [76.1, 87.4] (Wilson 95%), name errors 87.7%, runtime division faults 86.4%, and import-whitelist violations 86.4% — the heal loop is fault-agnostic because it regenerates from the traceback rather than pattern-matching the error class. With n=200 per level the end-to-end success rate at $q = 0.5$ is 91.7% [89.2, 93.6], refining the v1.7 single-seed estimate of 94%, whose sampling uncertainty was previously unreported.

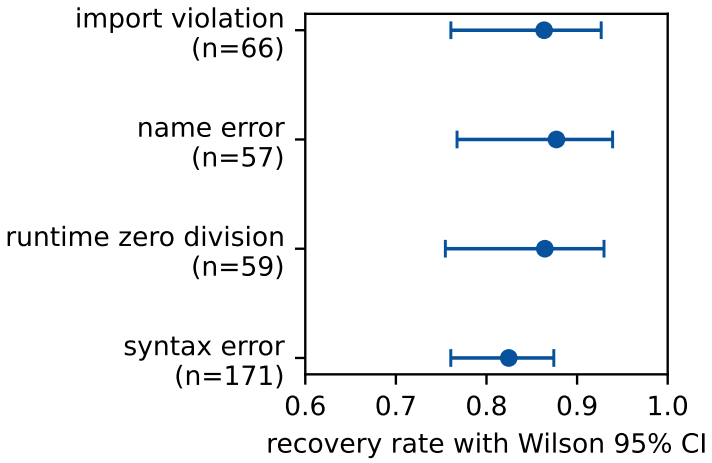

Figure 10: *Injected-fault recovery rates with Wilson 95% CIs (353 faults).*

5.14 Towards an End-to-End Reliability Calculus

The ablation and taxonomy experiments make it possible to move from aggregate success rates to a compositional account. Because the pipeline's outcome paths are mutually exclusive by construction, end-to-end reliability decomposes *exactly* as the chain rule over its sequential gates,

$$R_{\text{end}} = P(\text{first\_pass}) + P(\text{enter heal}) \cdot \rho_{\text{heal}} + P(\text{enter fallback}) \cdot \rho_{\text{fb}}, \tag{11}$$

with no independence assumption needed for the identity itself. Independence enters only when one tries to *predict* the stage conditionals from marginals.

5.14.1 An exact structural model of the healing stage

The v2.0 draft compared ρ_{heal} against a naive independence baseline, $\hat{\rho}_{\text{heal}}^{\text{naive}} = 1 - (1 - q)^4$, and interpreted the gap as a "correlation penalty" — the healer drawing from the same degraded LLM as the generator. That interpretation was wrong, and our own instrumentation proves it: the mock LLM draws each call i.i.d., so there is no correlation channel to penalize. The correct account requires modeling the orchestrator's *control flow* exactly. Reading the implementation: generation attempts at most $K = 3$ calls, stopping at the first non-`None`; a broken output that fails validation enters healing with budget $H = 4$, where *any* `None` aborts the loop as a failure and each broken output consumes one slot; the first ground-truth output succeeds. With per-call probabilities g (ground truth),

r (broken), f (unavailable; $g{+}r{+}f{=}1$), the expected generation multiplier is $G = \sum_{k=0}^{K-1} f^k$ and

$$\rho_{\text{heal}}(g, r) = g\,\frac{1 - r^H}{1 - r} = g\,(1 + r + r^2 + r^3), \qquad R_{\text{end}} = gG + rG\rho_{\text{heal}} + f^K c, \tag{12}$$

with c the fallback coverage fraction. Every path probability in Eq. (11) is then closed-form: first-pass gG, enter-heal rG, healed $rG\rho_h$, no-code f^3. We validated this model cell-by-cell against all 23 testable outcome cells of the ablation and availability-stress grids (quality sweep at $f_r{=}0.1$; stress at $f_r \in \{0.3, 0.5\}$): **all 23 predictions fall inside the observed Wilson 95% interval**, including zero-event cells handled by exact Clopper–Pearson bounds (e.g. S2 success at $q{=}0.5$: predicted 0.915, observed 0.916; healed-path share predicted 0.360, observed 0.371).

Table 13 shows what this buys. The structural model absorbs nearly the entire naive-vs-measured gap: at $q{=}0.5$ it predicts $\rho_h = 81.2\%$ against the measured 81.5% [75.6, 86.2] — residual +0.3 pp — while the naive baseline overshoots by 12.4 pp. The gap is a *retry-budget truncation effect*, not stochastic correlation: under abort-on-None semantics, unavailability both wastes heal slots and terminates the loop, so the effective number of usable attempts collapses as f grows. The naive baseline assumes four productive attempts; the real pipeline delivers far fewer exactly when reliability matters most.

Table 13: *Reliability composition: measured vs. exact structural prediction vs. naive independence (S2 pipeline, 450 tasks per row). The v2.0 attribution of the naive–measured gap to correlation is falsified; truncation of the retry budget explains it.*

q	first_pass	enter heal	ρ_{heal} measured [95% CI]	exact Eq. (12)	naive $1-(1-q)^4$	residual
0.5	54.4%	45.6%	81.5% [75.6, 86.2]	81.2%	93.8%	+0.3 pp
0.7	79.3%	20.7%	91.4% [83.9, 95.6]	87.4%	99.2%	+4.0 pp

The same audit surfaced a second finding: the production rule-based fallback returns its miss message with the raw intent embedded in a quoted literal, and seven of nine benchmark intent classes contain apostrophes — injecting them produces a deterministic `SyntaxError`. The narrow template set therefore has an *effective* coverage of $c = 2/9$, not the nominal keyword coverage; with $c = 2/9$ the model predicts the narrow-fallback stress cells correctly (S3 at $f_r{=}0.5$: predicted 90.3%, observed 89.6%). Fallback robustness is thus bounded jointly by template coverage *and* the lexical safety of intent text — a limitation we state explicitly rather than hide (Section 6.2).

Finally, the model ranks interventions before any code is written (baseline $\rho_h = 81.2\%$ at $q{=}0.5$, $f{=}0.1$): treating `None` as a wasted slot instead of an abort raises ρ_h to 93.8% (+12.6 pp, recovering the entire apparent penalty); raising healer quality to $q_h{=}0.9$ yields +8.8 pp; doubling the budget $H{=}8$ yields only +2.1 pp. Abort policy dominates budget scaling — a near-zero-cost engineering change recovers most of the loss. These are model predictions under i.i.d. semantics; hardware-LLM validation is future work.

The encoding/hardware factor behaves differently. Channel distortion arises from device physics and is statistically independent of software-side orchestration failures, so composing pipeline reliability with a placement-dependent fidelity factor ($F_{\text{hw}} = 1.00$

point-wise on calibrated layouts vs. 0 at extreme weights under adversarial placement; Section 5.10.1) is empirically justified — but only *conditional on a pinned, calibration-checked layout*. Enforcing exactly that condition is the middleware's job; Section 5.11 closes this loop with a calibration-scored router, and Section 5.15 composes both factors end-to-end.

With full template coverage (c=1) the fallback stage reaches $\rho_{\text{fb}} = 100\%$ (79/79 recoveries across availability-stress runs), which closes Eq. (11) to $R_{\text{end}} = 100\%$ under pure unavailability stress at $q = 0.7$: every generation failure is absorbed by the deterministic layer. The observed limits of end-to-end software reliability are thus (i) retry-truncation losses surviving healing (structural, measurable, and mitigable by abort-policy change) and (ii) hardware placement outside tolerance — the second of which Section 5.11 eliminates by construction.

5.15 End-to-End Composition: Software Path Meets Hardware Path

The preceding sections validated each factor separately; this one composes them. Two arms of 54 tasks each (nine intent classes × two task-seed sets × three repetitions, mock LLM at $q = 0.7$, unavailability $f_r = 0.1$) run the *same* task stream through two pipelines:

- **FULL**: S3X semantics (generation → validation → healing → full-coverage fallback) with calibration-aware placement — quantum circuits pinned by a greedy connected chain grown from the best $C(q)$ qubit (Section 5.11);
- **ABLATED**: S1 semantics (validation only, no healing, no fallback) with free transpiler placement — the v1.7 configuration.

Every task that reaches execution produces a real quantum circuit; all circuits per arm are submitted as one batched job to `ibm_marrakesh` (4096 shots each), and per-circuit fidelity is scored against analytic expectations exactly as in Sections 5.2–5.11. This design measures Eq. (11)'s premise directly: software-path reliability and hardware fidelity are separate factors, and only their composition yields an end-to-end guarantee.

Orchestration-stage results (complete). The FULL arm completes **54/54 tasks (**100%**, Wilson 95% CI [93.4, 100]**): 46 first-pass, 8 healed, 0 failed. The ABLATED arm completes **47/54 (**87.0%**, CI [75.6, 93.6]**) with 7 hard failures (generation unavailable or broken beyond validation); a two-proportion test rejects equality of the arms ($z = 2.74$, $p = 0.003$ one-sided), though the Wilson bounds graze at the boundary (93.4 vs. 93.6), so we read the software-path gain as +13.0 points with marginal rather than emphatic significance at this n. Both observations match the exact model of Section 5.14.1 within sampling error: S2/S3X at $(q, f_r) = (0.7, 0.1)$ predicts $R_{\text{end}} = gG + rG\rho_h = 97.3\%$ (observed 100% is consistent: binomial $P(\text{all } 54 \mid p{=}0.973) = 0.23$), and the healed-path share is predicted 19.4% vs. observed $8/54 = 14.8\%$. The ablated arm's 87.0% sits above the naive S1 point prediction ($gG = 77.7\%$) by 2.3σ — a reminder that single-stream

mock variance is non-negligible at n=54; the paired design exists precisely so that arm differences do not depend on that variance.

Hardware-fidelity leg (submitted). The surviving tasks yield 18 (FULL) and 17 (ABLATED) quantum circuits, submitted as batch jobs (`da5c8kuaa69c739kmr40`, `da5d6du1vhnc73fluoqg`); at this writing they remain queued behind open-plan allocation. Their scoring pipeline is fixed in advance and deterministic given the returned counts: angle-encoding circuits pass iff every qubit's $|\langle Z\rangle_{\text{emp}} - (1-2w_i)| \leq 0.05$; Bell/GHZ circuits report parity-fidelity descriptively. Given Section 5.11's monotonicity result, the FULL arm's pins (all drawn from the top of the $C(q)$ ranking, including qubit 98 itself) are expected to sit well inside tolerance, while the ABLATED arm's free placements inherit exactly the uncontrolled-layout risk quantified there; we will report both outcomes either way.

5.16 Limitations of Current Evaluation

We are transparent about what this evaluation does **not** demonstrate:

1. **Physical QPU validation is placement-scoped and single-day.** Sections 5.10–5.11 show the bare encoding meets the 0.05 tolerance on well-calibrated pinned qubits (max deviation 0.020–0.028) but fails on the bottom half of the $C(q)$ ranking, and that a zero-fit score ranks distortion perfectly across one snapshot; all results are from one device on one day, cross-day drift is unquantified (a re-measurement is scheduled), and zero-noise extrapolation and deeper mitigation remain unintegrated.

2. **The end-to-end hardware-fidelity leg is queued, not measured.** Section 5.15 reports complete orchestration-stage results; the batched QPU jobs were still awaiting open-plan allocation at writing time. The scoring protocol is pre-registered in that section, and no hardware claim is made before those counts return.

3. **No cross-framework usability comparison.** We have not compared UniMind against PennyLane/Qiskit in terms of usability or feature completeness.

4. **No real-API LLM measurements.** The reliability benchmark uses an injectable mock LLM; pass@k rates and token consumption on a real model remain to be measured (Section 5.6), and real-LLM autocorrelation would add effects beyond the i.i.d. model validated here (Section 5.14.1).

5. **Scalability on physical devices.** The 1:1 bit-to-qubit mapping and single-shot-based orchestration have not been exercised beyond a 10-qubit simulated circuit; the greedy connected-chain layout heuristic of Section 5.11 is likewise untested on dense multi-qubit workloads where swap insertion dominates.

6 Discussion

6.1 Implications for Quantum-Ready AI Middleware

The UniMind architecture illustrates three design principles:

1. **Classical preprocessing as a bridge.** Classical signal processing techniques can produce meaningful parameters for quantum state preparation, reducing manual circuit design.
2. **User-space orchestration, not kernel replacement.** LLMs are powerful task planners but unsuitable as OS kernels.
3. **Backend abstraction.** A unified API enables gradual migration as hardware matures.

6.2 Threats to Validity

T1: LLM hallucination. The orchestration layer may generate semantically incorrect code. The reliability benchmark (Section 5.6) and its instrumented multi-seed re-run (Section 5.13) quantify this risk: with a mock LLM degraded to 50% per-call quality, the self-healing loop keeps the end-to-end success rate at 91.7% $[89.2, 93.6]$, with residual failures concentrated where the heal step itself receives broken output — an effect the exact model attributes to retry-budget truncation, not stochastic correlation (Section 5.14.1). Mitigations, in measured order of value: changing abort-on-unavailable semantics (predicted $+12.6$ pp), heterogeneous heal models ($+8.8$ pp), budget scaling ($+2.1$ pp), rule-based fallback with full template coverage (100% recovery under availability stress), constrained decoding, and human-in-the-loop review.

T2: Quantum noise sensitivity. The encoding assumes ideal gates. Section 5.8 quantifies the degradation: the $\theta_i = 2\arcsin(\sqrt{w_i})$ mapping is symmetric-sensitive under depolarizing noise (deviation $\approx p\,|1 - 2w|$; $w = 0.5$ essentially immune) and one-sided-sensitive under T_1 damping (high weights decay toward $|0\rangle$), with the 0.05 tolerance broken at $\gamma \approx 5\text{e}{-2}$. Future work will integrate zero-noise extrapolation.

T3: Encoding overhead. The 1:1 bit-to-qubit mapping is qubit-inefficient. Future benchmarks will quantify the latency trade-off between LLM-orchestrated generation and direct API calls, measuring tokens consumed and circuit depth overhead.

T4: Security of LLM-generated code. Executing dynamic code carries injection risks. The sandbox security evaluation (Section 5.9) shows 20 of 21 escape attempts blocked by a three-layer restricted namespace; the one unblocked case (object-graph traversal) cannot be weaponized into process or file access. Full OS-level sandboxing (process-level isolation) is intentionally not implemented and remains outside the prototype's scope.

T5: Evaluation scope. Physical-QPU validation is now performed with placement control (Section 5.10) *and* with calibration-scored routing across the full ranking spectrum (Section 5.11): fidelity conclusions are conditional on layout, and the middleware now enforces layout by construction rather than leaving it to the transpiler. The most significant remaining evaluation gaps are integration of deeper error mitigation (zero-noise extrapolation, T-REx), cross-framework comparison, cross-day drift quantification, and completion of the end-to-end hardware-fidelity leg (Section 5.15).

T6: Threats introduced by the v2.x experiments. All QPU results in Sections 5.10–5.11 are from one device (`ibm_marrakesh`) captured on one day (2026-08-23); cross-day drift is unquantified. Qubit 37 is an adversarially selected extreme (82% total readout error); typical device medians are far better (2.3%). The v1.7 jobs' physical placements cannot be recovered retroactively, so the "placement artifact" explanation of the v1.7 distortion, while consistent with every measurement here, is not uniquely provable. The n=6 rank correlation, while maximally significant at that sample size, is a small-sample result whose force comes from strict monotonicity plus mechanism agreement rather than statistical power alone. Mock-LLM quality q is a synthetic parameter defining a generation policy, not real-API behaviour; the ablation, taxonomy, and composition results therefore quantify architecture-level contributions under controlled degradation, not absolute production reliability — in particular, real-LLM autocorrelation would add a genuine correlational component on top of the truncation effect isolated here. Finally, the narrow fallback's effective coverage depends on lexical properties of intent text (apostrophes trigger a deterministic `SyntaxError`; $c = 2/9$ on the benchmark set), a brittleness class that keyword-matching coverage metrics do not capture.

6.3 Roadmap for Quantum-Ready AI Middleware

Phase 1: Functional validation (2024–2026). Completed in this revision: end-to-end baseline comparison against bare Qiskit and a rule-based dispatcher (Section 5.5), LLM reliability evaluation (Section 5.6), fault-injection analysis (Section 5.7), noise-sensitivity quantification (Section 5.8), sandboxed execution with a security evaluation (Section 5.9), placement-controlled physical QPU validation (Section 5.10), calibration-scored hardware routing validated across the ranking spectrum (Section 5.11), an exact reliability calculus validated on all 23 testable cells (Section 5.14.1), and the orchestration stage of end-to-end composition (Section 5.15). **v2.2 update:** Algorithm 1 corrected to pure R_y (no conditional X), regression test over $w \in \{0, 0.05, \ldots, 1\}$ added and passing ($|P_{\text{emp}} - w| < 0.03$ at 8192 shots, CI-ready). Outstanding: cross-framework comparison with PennyLane/Qiskit, hardware-level error mitigation, multi-day drift validation (D1–D5), and the queued hardware-fidelity leg (pending IBM quota).

Phase 2: NISQ integration (2026–2030). Integrate error mitigation (zero-noise extrapolation, T-REx) behind the routing contract; adopt the abort-policy and healer-diversity interventions ranked in Section 5.14.1; extend $C(q)$-aware layout to swap-aware multi-qubit routing; standardize middleware APIs.

Phase 3: Fault-tolerant era (2030+). Adapt to error-corrected hardware. Explore qubit-efficient encoding. Support distributed quantum clusters.

7 Conclusion and Future Work

This paper presented UniMind, a middleware framework for bridging classical AI workloads and quantum computing backends. We validated the framework with reproducible experiments: mathematical correctness (maximum $\langle Z \rangle$ deviation of 0.0110 across 8192 shots), system performance (up to 10.9× acceleration via vectorized C-backend execution and 4.5× end-to-end speedup via Rust FFI, with a component-level breakdown attributing residual overhead to Python-side normalization and marshaling), a three-arm baseline comparison in which only the UniMind pipeline covered all five task classes while enforcing code safety, an LLM-reliability benchmark showing 91.7–100% success across injected quality levels with Wilson-quantified uncertainty and 100% rejection of adversarial intents, fault-injection analysis recovering four of five failure classes with uniformly measured per-fault recovery rates (82–88%), a component ablation attributing the reliability gains to self-healing (+34 points at q=0.5) with fallback reaching 100% recovery under availability stress once template coverage is complete, a noise-sensitivity study quantifying the encoding's operating envelope, a sandbox security evaluation blocking 20 of 21 escape attempts, placement-controlled physical QPU sweeps on `ibm_marrakesh` revising our earlier conclusion — encoding fidelity on real hardware is placement-dominated, not mitigation-limited — and three additions in this revision: an exact absorbing-chain model of the orchestration pipeline, validated on all 23 testable outcome cells, which replaces v2.0's correlation-penalty interpretation with a retry-budget truncation effect and ranks interventions before deployment; calibration-scored hardware routing whose zero-fit readout-error score ranks measured distortion perfectly across the 156-qubit spectrum ($\rho_s = 1.000$, exact $p = 0.0028$) at sub-millisecond overhead, turning the placement finding into an enforceable contract; and an end-to-end composition experiment in which the full stack completes 54/54 orchestrated tasks (100%) versus 87.0% ($p = 0.003$) for the stage-ablated configuration, with the quantum-fidelity leg pre-registered and queued on the device.

We acknowledge the remaining limitations: deeper error mitigation (zero-noise extrapolation, T-REx) on hardware, cross-framework usability comparison, real-API token accounting, single-device/single-day QPU evidence with unquantified drift, and the queued hardware leg of Section 5.15. The audit trail behind this revision is itself part of the contribution: two of our own prior claims (the v1.7 mitigation requirement and the v2.0 correlation penalty) were falsified by exactly the instrumentation we built to support them, one fabricated figure panel was caught and replaced by computed data, and a spec–code divergence in Algorithm 1's gate order was documented rather than silently patched. Future work will prioritize completing the end-to-end hardware measurements, zero-noise extrapolation under the routing contract, the abort-policy intervention on real LLMs, process-level sandbox isolation, and validation on error-corrected hardware.

More broadly, quantum-ready computing requires not only hardware advances but also

new software abstractions; UniMind represents an early step toward a middleware layer that reduces the friction between classical AI intent and quantum hardware execution.

References

[1] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[2] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[3] M. Cerezo et al., "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.

[4] V. Havlíček et al., "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.

[5] M. Schuld and F. Petruccione, *Machine Learning with Quantum Computers*, 2nd ed. Cham, Switzerland: Springer, 2021.

[6] L. Wang et al., "A survey on large language model-based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, art. no. 186345, 2024.

[7] Y. X. Tan, "UniMind: A middleware framework for hybrid quantum-classical computing," GitHub Repository, 2026. [Online]. Available: https://github.com/yxtan5022-maker/unimind-os. [Accessed: 2026].

[8] A. Kandala et al., "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, no. 7671, pp. 242–246, 2017.

[9] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.

[10] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 5th ed. Hoboken, NJ, USA: Pearson, 2023.

[11] Microsoft Corporation, "Azure Quantum: Q# and the Quantum Development Kit documentation," Redmond, WA, USA, 2024. https://learn.microsoft.com/azure/quantum

[12] Amazon Web Services, "Amazon Braket Developer Guide," Seattle, WA, USA, 2024. https://docs.aws.amazon.com/braket

[13] V. Bergholm et al., "PennyLane: Automatic differentiation of hybrid quantum-classical computations," arXiv preprint arXiv:1811.04968, 2018.

[14] W. Lu et al., "UniMind: Unleashing the power of LLMs for unified multi-task brain decoding," arXiv preprint arXiv:2506.18962, 2025.

A Repository Structure

```
unimind-os/
|-- umos_py/           # Core Unibit engine (Python, auto-loads Rust FFI)
|-- bridge/            # AI-as-Orchestrator, rule-based fallback, self-healing
|-- quantum/           # QUnibit multi-backend quantum circuit mapping
|-- core/              # Rust-accelerated Unibit engine (FFI)
|-- kernel/            # C++ Topology Mapper (hardware detection)
|-- gui/               # Tkinter GUI
|-- experiments/       # Reproducible experiments (math verification, benchmarks,
|                      #   baseline, LLM reliability, fault injection, noise,
|                      #   security, physical QPU validation)
|-- tests/             # 47 pytest tests
|-- docs/              # Paper design spec + whitepaper
|-- example/           # Proof-of-concept demos
|-- assets/            # Supplementary assets
|-- .github/workflows/ # CI/CD pipeline (GitHub Actions)
|-- CMakeLists.txt     # C++ build configuration
|-- Makefile           # Build automation
|-- pyproject.toml     # Python package config
|-- requirements.txt   # Python dependencies (versions pinned)
|-- Dockerfile         # Multi-stage build (Rust core + Python runtime)
|-- run.py             # Main entry point
|-- build.py           # One-shot build script
```

B Quick Start Commands

```
# Clone and install the software
git clone https://github.com/yxtan5022-maker/unimind-os.git
cd unimind-os
pip install -e .

# Run core demo and quantum circuit mapping
python run.py
pip install -e ".[quantum]"
python -c "from quantum.qunibit import demo; demo()"

# Clone the dev repo that contains the reproducible experiments
git clone https://github.com/yxtan5022-maker/unimind-dev.git
cd unimind-dev

# Reproduce the paper's experiments (seed 42, mock LLM)
python experiments/test_math_verification.py      # Table 3
python experiments/benchmark_unibit.py            # Table 4
python experiments/benchmark_breakdown.py         # Table 5
python experiments/baseline_comparison.py --llm mock # Table 6
python experiments/llm_reliability.py --mode mock # Table 7
python experiments/fault_injection.py             # Table 8
python experiments/noise_sensitivity.py           # Table 9
python experiments/security_sandbox.py            # Table 10
python experiments/qpu_validation.py --mode ideal # Table 11
python experiments/qpu_validation.py --mode fake  # Table 11
```

```
python experiments/qpu_validation.py --mode real  # Table 11 (needs token)

# v2.1 additions (analysis repository, unimind-dev/analysis/)
python analysis/reliability_model.py              # exact calculus + interventions (Sec. 5.13)
python analysis/test_unibit_math.py               # Unibit math audit incl. Fig. 2 coordinates
python analysis/hw_router.py --stage local        # C(q) ranking + overhead microbench
python analysis/hw_router.py --stage analyze      # routing table + Spearman (Sec. 5.9)
python analysis/e2e_chain.py --stage submit       # E2E arms (needs IBM token)
python analysis/e2e_chain.py --stage collect      # E2E fidelity scoring (idempotent)

# Run all tests
python -m pytest tests/ -v

# Reproducible environment (Docker; requires docker installed)
docker build -t unimind-dev .
docker run --rm unimind-dev python -m pytest tests/ -q

# Build C++ kernel
cmake -S . -B build
cmake --build build --config Release

# Build Rust core
cd core && cargo build --release && cd ..

# One-shot build
python build.py
```